

Natural Language Processing (AI408DV01)

Bài tập thực hành cá nhân – Tuần 02

Name:	Nguyễn Phúc Minh Châu
Student ID:	22302421
Deadline:	23/03/2026 - 30/03/2026
File name format:	AI301DV01-Lab-02-22302421.pdf

A. Chuẩn bị

- Đọc kỹ slide bài giảng **Word Tokenization**
- [1] Chapter 2. Dan Jurafsky and James H. Martin, *Speech and Language Processing*, 3rd ed. Pearson, 2026.
- [3] Chapter 2. Jay Alamar and Maarten Grootendorst, *Hands-On Large Language Models*. O'Reilly, 2024.

B. Câu hỏi lý thuyết

Bài tập 1.

Tại sao phương pháp phân đoạn thành "token con" (subword tokens) lại có ưu điểm hơn so với phân đoạn theo "từ" (word tokens) hoặc "ký tự" (character tokens)?

Phương pháp phân đoạn thành **token con (subword tokens)** được coi là giải pháp tối ưu và cân bằng giữa phân đoạn theo từ và theo ký tự trong các mô hình ngôn ngữ hiện đại vì những lý do sau:

1. Ưu điểm so với phân đoạn theo từ (Word tokens)

a. Giải quyết vấn đề từ lạ (Out-of-Vocabulary - OOV):

Nhược điểm lớn nhất của phân đoạn theo từ là các mô hình dựa trên từ không thể xử lý những từ mới không xuất hiện trong tập huấn luyện, mà thường phải thay thế chúng bằng token đặc biệt **<UNK>**. Token con giải quyết điều này bằng cách chia các từ lạ thành các mảnh nhỏ hơn đã tồn tại trong từ vựng (ví dụ: **"lower"** có thể được tách thành **"low"** và **"er"**).

b. Giảm sự dư thừa và tăng tính biểu đạt của từ vựng:

Phân đoạn theo từ dẫn đến việc từ vựng chứa nhiều từ có gốc giống nhau nhưng hậu tố khác nhau (như *"apology"*, *"apologize"*, *"apologetic"*). Token con cho phép mô hình học các thành phần mang nghĩa (hình vị) như tiền tố và hậu tố, giúp từ vựng gọn nhẹ nhưng vẫn diễn đạt được nhiều biến thể từ khác nhau.

c. Xử lý tốt các ngôn ngữ không có khoảng trắng:

Nhiều ngôn ngữ như tiếng Trung, tiếng Nhật hoặc tiếng Thái không sử dụng khoảng trắng để phân tách từ, khiến việc định nghĩa "từ" một cách chính thức trở nên khó khăn.

2. Ưu điểm so với phân đoạn theo ký tự (Character tokens)

a. Hiệu quả về độ dài chuỗi và tài nguyên tính toán:

Việc phân đoạn theo ký tự khiến độ dài chuỗi token tăng lên đáng kể (ví dụ: một câu có **13 từ** có thể biến thành **81 ký tự**). Điều này làm chậm quá trình huấn luyện và suy luận, đồng thời chiếm dụng nhiều không gian trong "context length" (độ dài ngữ cảnh) hạn chế của các mô hình Transformer. Token con giúp nén thông tin hiệu quả hơn, cho phép đưa nhiều nội dung hơn vào một cửa sổ ngữ cảnh.

b. Giảm độ phức tạp của mô hình:

Khi sử dụng ký tự, mô hình phải học cách "đánh vần" từng từ trước khi hiểu được ý nghĩa của câu. Với token con, nhiều từ thông dụng vẫn được giữ nguyên dưới dạng một token duy nhất, giúp mô hình tập trung vào việc học các mối quan hệ cú pháp và ngữ nghĩa phức tạp thay vì chỉ học cấu trúc từ.

Phương pháp token con cho phép các mô hình ngôn ngữ duy trì một bộ từ vựng có kích thước hợp lý (thường từ 50.000 đến 200.000 token), vừa có khả năng xử lý linh hoạt mọi từ ngữ mới, vừa đảm bảo hiệu suất tính toán cao.

Bài tập 2.

Các yếu tố chính quyết định cách một bộ tokenizer phân đoạn văn bản đầu vào?

Cách một bộ tokenizer phân đoạn văn bản đầu vào được quyết định bởi ba yếu tố chính sau đây:

- **Phương pháp tách từ (Tokenization Method):**

Đây là thuật toán cốt lõi được chọn ở giai đoạn thiết kế mô hình. Các phương pháp phổ biến bao gồm **Byte-Pair Encoding (BPE)** (thường dùng cho GPT), **WordPiece** (dùng cho BERT), hoặc **Unigram Language Modeling (ULM)**. Mỗi phương pháp có cách tiếp cận khác nhau để tối ưu hóa tập hợp các token nhằm biểu diễn dữ liệu văn bản một cách hiệu quả nhất.

Ví dụ, BPE dựa trên tần suất xuất hiện của các cặp ký tự liền kề để hợp nhất chúng, trong khi WordPiece sử dụng phương pháp xác suất (likelihood) để quyết định việc hợp nhất.

- **Các tham số thiết kế (Tokenizer Parameters):**

Sau khi chọn phương pháp, các nhà thiết kế mô hình phải đưa ra các quyết định quan trọng về tham số:

- **Kích thước từ vựng (Vocabulary size):**

Số lượng token tối đa mà bộ tokenizer được phép giữ trong từ vựng (ví dụ: 30.000, 50.000 hoặc lên tới 200.000 token). Kích thước lớn giúp biểu diễn văn bản bằng ít token hơn nhưng có thể dẫn đến việc các token hiếm gặp có biểu diễn kém.

- **Các token đặc biệt (Special tokens):**

Việc bổ sung các token có vai trò cụ thể như bắt đầu văn bản (`<s>`), kết thúc văn bản (`</s>`), token đệm (`[PAD]`), hoặc token dùng cho phân loại (`[CLS]`).

- **Xử lý chữ hoa/chữ thường (Capitalization):**

Quyết định xem có chuyển tất cả về chữ thường (**uncased**) hay giữ nguyên sự khác biệt (**cased**). Việc giữ nguyên chữ hoa chữ thường thường tốt hơn vì nó chứa đựng thông tin ngữ nghĩa quan trọng.

- **Tập dữ liệu huấn luyện (Training Dataset):**

Bộ tokenizer cần được huấn luyện trên một tập dữ liệu cụ thể để thiết lập từ vựng tối ưu. Ngay cả khi sử dụng cùng một phương pháp và

tham số, một bộ tokenizer được huấn luyện trên tiếng Anh sẽ phân đoạn văn bản khác hoàn toàn với bộ được huấn luyện trên mã nguồn (code) hoặc đa ngôn ngữ.

Ví dụ, các bộ tokenizer đa ngôn ngữ thường ưu tiên tiếng Anh (do dữ liệu huấn luyện chiếm ưu thế), dẫn đến việc các ngôn ngữ khác bị chia nhỏ thành nhiều token ngắn hơn, làm giảm hiệu quả xử lý.

Ngoài ra, **quy trình tiền mã hóa (pre-tokenization)** cũng đóng vai trò quan trọng, trong đó các **biểu thức chính quy (regular expressions)** được sử dụng để phân tách văn bản thô theo khoảng trắng, dấu câu, hoặc xử lý các nhóm chữ số trước khi thuật toán tách từ con thực hiện nhiệm vụ của mình.

Bài tập 3.

Cách một bộ tokenizer xử lý khoảng trắng (spaces) và chữ số (digits)?

1. Xử lý Khoảng trắng (Spaces)

Khoảng trắng thường được coi là ranh giới cơ bản để phân đoạn văn bản, nhưng cách các bộ tokenizer hiện đại lưu trữ chúng rất đa dạng:

- **Dùng làm ký tự phân tách:** Trong các hệ thống cũ hoặc đơn giản, khoảng trắng chỉ dùng để chia văn bản thành các từ và sau đó bị loại bỏ.

- **Gắn vào đầu từ (Pre-tokenization):**

Các bộ tokenizer như của GPT thường sử dụng **biểu thức chính quy (regex)** để tách thô văn bản theo khoảng trắng trước khi chạy thuật toán subword. Khoảng trắng thường được gắn vào đầu của mỗi từ và được thay thế bằng một ký tự đặc biệt để mô hình có thể tái tạo lại văn bản gốc chính xác. (Ví dụ: **Ġ** trong *GPT* hoặc **_** trong *SentencePiece*)

- **Mã hóa nhiều khoảng trắng:**

Các mô hình hiện đại như GPT-4 có khả năng nhóm nhiều khoảng trắng liên tiếp thành một token duy nhất (lên đến 83 khoảng trắng) để tăng hiệu suất xử lý, đặc biệt hữu ích trong việc lập trình (code) nơi khoảng trắng dùng để thụt lề.

- **Xử lý xuyên ranh giới từ:**

Một số thuật toán nâng cao như SuperBPE cho phép các token trải

rộng qua nhiều từ, nghĩa là chúng có thể kết hợp cả khoảng trắng và dấu câu vào bên trong một token để đạt hiệu quả nén cao hơn.

2. Xử lý Chữ số (Digits)

Việc xử lý con số là một thách thức vì số lượng các con số là vô hạn, không thể đưa tất cả vào từ vựng:

- **Chia nhóm chữ số (Chunking):**

Nhiều bộ tokenizer (như GPT-4o) sẽ chia các số dài thành từng nhóm nhỏ, thường là 3 chữ số một nhóm (ví dụ: "224123" có thể tách thành '224' và '123').

- **Tách từng chữ số đơn lẻ:**

Để hỗ trợ tốt hơn cho các bài toán toán học và lập trình, một số bộ tokenizer (như Galactica hoặc StarCoder2) quy định mỗi chữ số từ 0-9 là một token riêng biệt. Cách tiếp cận này giúp mô hình tránh bị nhầm lẫn khi gặp các tổ hợp số hiếm gặp.

- **Phân đoạn thành token con (Subwords):**

Nếu một con số không có sẵn trong từ vựng và không được xử lý theo quy tắc chữ số đơn lẻ, nó sẽ bị tách thành các phần nhỏ hơn dựa trên tần suất xuất hiện trong dữ liệu huấn luyện (ví dụ: số 937 có thể bị tách thành '9' và '37').

- **Sử dụng biểu thức chính quy:**

Trong giai đoạn tiền xử lý, các bộ tokenizer thường dùng regex (như `\p{N}+`) để nhận diện và tách các chuỗi chữ số ra khỏi các ký tự chữ cái hoặc dấu câu.

Bài tập 4. (xem tài liệu số 2)

So sánh các bộ LLM tokenizer được huấn luyện (Trained LLM Tokenizers).

Việc so sánh các bộ tokenizer của các **mô hình ngôn ngữ lớn (LLM)** đã được huấn luyện dựa trên ba yếu tố chính: thuật toán tách từ, các tham số thiết kế (như kích thước từ vựng và mã thông báo đặc biệt), và **ngữ liệu (dataset)** mà

bộ tokenizer đó được huấn luyện. Dưới đây là bảng so sánh chi tiết các bộ tokenizer phổ biến:

1. So sánh theo thuật toán và đặc điểm kỹ thuật

Bộ Tokenizer (Mô hình)	Thuật toán chính	Kích thước từ vựng (Vocab Size)	Đặc điểm nổi bật
BERT	WordPiece	~30.000 (tiếng Anh)	Có hai phiên bản: Cased (phân biệt hoa thường) và Uncased (không phân biệt); sử dụng các mã đặc biệt [CLS], [SEP], [MASK], [PAD], [UNK].
GPT-2	Byte Pair Encoding (BPE)	50.257	Chạy trên các byte thay vì ký tự Unicode để tránh lỗi từ ngoài từ điển (OOV); bảo toàn khoảng trắng và xuống dòng.
GPT-4/GPT-4o	Byte Pair Encoding (BPE)	~100.000 (GPT-4) / 200.000 (GPT-4o)	Hiệu quả hơn trong việc nén văn bản; có các mã đặc biệt cho lập trình (như <code>elif</code>) và xử lý tốt các chuỗi khoảng trắng dài (lên tới 83 khoảng trắng liên tiếp).
Flan-T5	Unigram (SentencePiece)	32.100	Không bảo toàn xuống dòng hoặc khoảng trắng dư thừa; thay thế emoji và ký tự lạ bằng mã <code><unk></code> .
StarCoder 2	Byte Pair Encoding (BPE)	49.152	Tối ưu cho lập trình; mỗi chữ số được gán một token riêng (ví dụ: 600 thành 6, 0, 0) để cải thiện khả năng tính toán và xử lý số học.
Llama2/Phi-3	Byte Pair Encoding (BPE)	32.000	Bổ sung các mã hội thoại đặc biệt như <code>`<</code>

2. Các điểm khác biệt cốt lõi

Xử lý ngôn ngữ khác ngoài tiếng Anh: Các bộ tokenizer đa ngôn ngữ thường ưu tiên tiếng Anh vì dữ liệu huấn luyện chiếm đa số. Ví dụ, cùng một câu tiếng Tây Ban Nha có thể bị tách thành 33 tokens, trong khi bản dịch tiếng Anh tương đương chỉ mất 18 tokens. Việc **chia quá nhỏ (oversegmenting)** ở các ngôn ngữ ít nguồn lực có thể **dẫn đến việc biểu diễn ngữ nghĩa kém và tăng chi phí tính toán**.

Xử lý khoảng trắng và mã lập trình: Các tokenizer hiện đại (như GPT-4, Galactica) gán các token duy nhất cho các chuỗi khoảng trắng hoặc tab có độ dài

khác nhau để hỗ trợ thực đầu dòng trong lập trình. Ngược lại, các tokenizer cũ như Flan-T5 thường bỏ qua thông tin này.

- **Khả năng xử lý từ ngoài từ điển (OOV):**

WordPiece (BERT) nếu gặp ký tự không có trong bộ huấn luyện, nó sẽ trả về mã [UNK].

- **Byte-level BPE (GPT):**

Sử dụng 256 giá trị byte cơ bản làm nền tảng, đảm bảo mọi ký tự Unicode đều có thể được mã hóa bằng cách kết hợp các byte này, loại bỏ hoàn toàn vấn đề OOV.

Mã đặc biệt cho các lĩnh vực chuyên biệt:

Bộ tokenizer của Galactica (chuyên về khoa học) có các mã riêng cho trích dẫn tài liệu ([START_REF]), chuỗi DNA, và các bước suy luận từng bước (<work>).

3. Đánh giá hiệu quả (Metrics)

Để so sánh hiệu suất giữa các bộ tokenizer, người ta thường sử dụng hai chỉ số chính:

- **Fertility (Độ màu mỡ):**

Tỷ lệ trung bình số token cần thiết để biểu thị một từ. Chỉ số này càng cao thì khả năng nén của tokenizer càng kém.

- **Parity (Tính bình đẳng):**

So sánh số lượng token cần thiết để biểu thị cùng một nội dung giữa hai ngôn ngữ khác nhau. Một tokenizer tốt nên có tỷ lệ **tính chẵn lẻ (parity)** gần bằng 1 giữa các ngôn ngữ chính.

Xu hướng của các bộ tokenizer huấn luyện mới nhất là tăng kích thước từ vựng, sử dụng thuật toán byte-level BPE để hỗ trợ đa ngôn ngữ/emoji tốt hơn, và tối ưu hóa việc tách số/khoảng trắng để phục vụ các tác vụ toán học và lập trình.